

Dynamic Query em Service Builder

#programming

#liferay

Tutorial oficial - Using Dynamic Query.

No seu projeto, pense em **Dynamic Query** como uma forma de montar consultas ao banco usando os atributos das entidades Java, em vez de escrever SQL diretamente. A documentação do Liferay explica que a API envolve a Criteria API do Hibernate e trabalha com objetos e seus membros.

Há, porém, uma observação importante: **na documentação atual do Liferay 7.4+, DSL Query é o método recomendado para novas queries customizadas de entidades Service Builder.** Dynamic Query continua sendo uma API existente e útil para entender o Service Builder e código legado.

1. Por que Dynamic Query existe?

Você já tem no seu projeto coisas como:

```
h7g5EntryPersistence.findByName(name);
```

Isso é ótimo quando a consulta é simples e você consegue descrevê-la com um `<finder>` no `service.xml`.

Mas imagine que você queira:

```
"Pegue todos os H7G5Entry cujo nome contenha 'Java', cuja descrição não seja nula, que pertençam a determinada pasta e ordene pelo nome."
```

Um finder simples começa a ficar inadequado.

Você poderia escrever SQL, mas Dynamic Query permite construir a consulta programaticamente:

```
H7G5Entry
├──
├── condição 1
├── condição 2
├── condição 3
└── ordenação
```

↓
resultado

A documentação apresenta Dynamic Query justamente como uma alternativa para consultas mais complexas que simples finders.

2. O conceito principal: `DynamicQuery`

Você começa criando uma consulta:

```
DynamicQuery query =  
    DynamicQueryFactoryUtil.forClass(  
        H7G5Entry.class,  
        getClass().getClassLoader()  
    );
```

Você está dizendo:

```
"Quero construir uma consulta para a entidade H7G5Entry."
```

Não está executando nada ainda.

É como construir:

```
query = "quero consultar H7G5Entry"
```

Depois você vai acrescentando condições.

A API é **encadeável**, ou seja, você pode adicionar critérios sucessivamente.

3. `Restrictions`: o equivalente ao `WHERE`

Esse é provavelmente o conceito mais importante.

A documentação compara as `Restrictions` às condições de um `WHERE` SQL.

Por exemplo:

```
query.add(  
    RestrictionsFactoryUtil.eq("name", "Walter")  
);
```

Conceitualmente:

```
WHERE name = 'Walter'
```

0:

```
eq()
```

significa **equal**.

Existem vários operadores:

```
eq()      // =  
ne()      // !=  
gt()      // >  
ge()      // >=  
lt()      // <  
le()      // <=  
like()    // LIKE  
in()      // IN  
between()
```

A documentação lista esses operadores explicitamente.

4. Aplicando ao seu `H7G5Entry`

Suponha que você queira:

```
| Buscar entradas cujo name seja "Java".
```

Você poderia fazer:

```
DynamicQuery query =  
    DynamicQueryFactoryUtil.forClass(  
        H7G5Entry.class,  
        getClass().getClassLoader()  
    );  
  
query.add(  
    RestrictionsFactoryUtil.eq("name", "Java")  
);
```

Conceitualmente:

```
SELECT *  
FROM OHQIWTSFHL_H7G5Entry  
WHERE name = 'Java';
```

A diferença é que você não está trabalhando diretamente com:

```
tabela → coluna
```

mas com:

```
classe → atributo
```

A documentação destaca exatamente essa diferença.

5. Várias condições

Agora podemos fazer algo mais interessante.

Imagine:

```
h7g5FolderId = 10  
E  
name = "Java"
```

```
query.add(  
    RestrictionsFactoryUtil.eq(  
        "h7g5FolderId", 10L  
    )  
);  
  
query.add(  
    RestrictionsFactoryUtil.eq(  
        "name", "Java"  
    )  
);
```

Conceitualmente:

```
WHERE h7g5FolderId = 10  
AND name = 'Java'
```

É aqui que Dynamic Query começa a ser mais interessante do que um finder simples.

6. `like()`

Imagine que você queira procurar nomes que **contenham** determinada palavra:

```
query.add(  
    RestrictionsFactoryUtil.like(  
        "name", "%Java%"  
    )  
);
```

Conceitualmente:

```
WHERE name LIKE '%Java%'
```

Isso permitiria encontrar:

```
Java  
Java Programming  
Aprendendo Java  
Curso de Java
```

7. Executando a consulta

Até aqui você apenas **montou** a query.

Para executá-la, no contexto do seu `LocalServiceImpl`, você pode usar:

```
return dynamicQuery(query);
```

A documentação mostra exatamente esse padrão.

Por exemplo:

```
public List<H7G5Entry> getEntriesByName(String name) {  
  
    DynamicQuery query =  
        DynamicQueryFactoryUtil.forClass(  
            H7G5Entry.class,
```

```

        getClass().getClassLoader()
    );

    query.add(
        RestrictionsFactoryUtil.eq("name", name)
    );

    return dynamicQuery(query);
}

```

O fluxo é:

```

getEntriesByName()
    ↓
cria DynamicQuery
    ↓
adiciona Restrictions
    ↓
dynamicQuery(query)
    ↓
Persistence
    ↓
Hibernate
    ↓
Banco
    ↓
List<H7G5Entry>

```

8. Ordenação

Você também pode controlar a ordem.

A documentação usa:

```
Order order = OrderFactoryUtil.desc("name");
```

para ordenar pelo `name` em ordem decrescente.

Por exemplo:

```

query.addOrder(
    OrderFactoryUtil.asc("name")
);

```

Conceitualmente:

```
ORDER BY name ASC
```

ou:

```
query.addOrder(  
    OrderFactoryUtil.desc("name")  
);
```

equivale a:

```
ORDER BY name DESC
```

9. Projection

Esse conceito é um pouco diferente.

Normalmente:

```
dynamicQuery(query)
```

retorna objetos:

```
List<H7G5Entry>
```

Mas você pode dizer:

```
"Não quero os objetos completos. Quero somente determinado atributo."
```

A documentação chama isso de **projection**.

Por exemplo:

```
query.setProjection(  
    PropertyFactoryUtil.forName("name")  
);
```

Em vez de:

```
H7G5Entry  
H7G5Entry  
H7G5Entry
```

you can obtain something conceptually like:

```
"Java"  
"Python"  
"SQL"
```

There are also projections for operations like:

```
MAX  
MIN  
SUM  
AVG
```

as per the documentation.

10. A complete example for your project

Imagine that you want to create:

```
getEntriesByFolderAndName()
```

that searches `H7G5Entry`:

- within a determined folder;
- whose name contains a word;
- ordered by name.

In your:

```
h7g5-service/  
└─ H7G5EntryLocalServiceImpl.java
```

could be conceptually:

```
@Override  
public List<H7G5Entry> getEntriesByFolderAndName(  
    long h7g5FolderId, String name) {  
  
    DynamicQuery query =  
        DynamicQueryFactoryUtil.forClass(  
            H7G5Entry.class,  
            getClass().getClassLoader()  
        );
```



```

        query.add(
            RestrictionsFactoryUtil.eq(
                "h7g5FolderId", h7g5FolderId
            )
        );

        query.add(
            RestrictionsFactoryUtil.like(
                "name", "%" + name + "%"
            )
        );

        query.addOrder(
            OrderFactoryUtil.asc("name")
        );

        return dynamicQuery(query);
    }

```

A consulta conceitualmente seria:

```

SELECT *
FROM OHQIWTSFHL_H7G5Entry
WHERE h7g5FolderId = ?
      AND name LIKE ?
ORDER BY name ASC;

```

Mas você escreveu a consulta usando:

```

H7G5Entry
  ↓
"h7g5FolderId"
"name"
  ↓
Restrictions
  ↓
Order

```

em vez de escrever SQL.

11. Como isso se relaciona ao seu `service.xml`

Você já conhece os finders:

```

<finder name="Name" return-type="H7G5Entry">
  <finder-column name="name" />

```

```
</finder>
```

Isso gera algo como:

```
h7g5EntryPersistence.findByName(name);
```

Para uma consulta simples:

```
"WHERE name = X"
```

Finder é melhor e mais simples.

Dynamic Query entra quando a consulta começa a exigir mais composição:

```
WHERE  
  condição A  
  AND condição B  
  AND condição C  
ORDER BY ...
```

ou quando você precisa de projections etc.

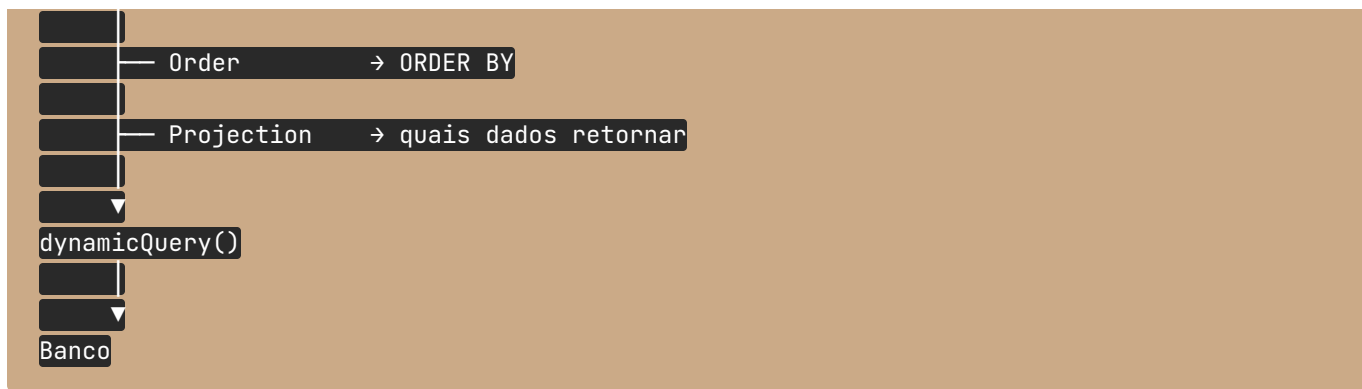
12. O que você deve guardar

Se você está estudando Service Builder, eu reduziria Dynamic Query a estes **5 conceitos**:

Conceito	Função
<code>DynamicQuery</code>	Representa a consulta que você está construindo
<code>DynamicQueryFactoryUtil</code>	Cria a consulta para uma entidade
<code>RestrictionsFactoryUtil</code>	Adiciona condições, equivalente ao <code>WHERE</code>
<code>OrderFactoryUtil</code>	Define <code>ORDER BY</code>
<code>dynamicQuery(query)</code>	Executa a consulta

Mentalmente:

```
DynamicQuery  
└──  
    └── Restrictions → WHERE
```



E há uma ressalva importante para o seu projeto: **não use Dynamic Query simplesmente porque ela parece mais poderosa que findBy...** Para uma busca que seu `service.xml` consegue representar adequadamente, o finder é mais direto. E para um projeto novo em Liferay 7.4+, a própria documentação atual recomenda **DSL Query** para custom queries de entidades Service Builder.